SHRI RAM SWAROOP MEMORIAL UNIVERSITY

Deva Road, Lucknow, Uttar Pradesh

PROJECT REPORT ON

DIGITAL ELECTRONICS LOGIC GATE SIMULATOR

A Self-Contained Interactive Digital Systems Learning Tool

Submitted To:

Dr. Prabhat Srivastava

Submitted By:

S.No.	Student Name	Roll Number
1	**Manas Jaiswal**	202510101180029
2	**Harshit Singh**	202510101180001
3	**Sachin Yadav**	202510101180021
4	**Suraj Singh**	202510101180020
5	**Vivek Singh**	202510101180011

PROJECT REPORT | DIGITAL LOGIC LEARNING TOOL

Digital Electronics Logic Gate Simulator

A three-page project report

Executive summary

The Digital Electronics Logic Gate Simulator is a self-contained teaching application that lets a learner change binary inputs and immediately inspect the resulting logic. It places the circuit symbol, Boolean rule, live output, and truth table together, making the relationship between a gate’ s definition and its behavior easy to follow. Eight gates and five ready-made circuits are supported, including stateful examples. A Learn section, short quiz, and CSV export extend the app from demonstration to practice. It runs from local files with plain HTML, CSS, and JavaScript.

8	5	10	0
logic gates	circuit models	quiz questions	runtime dependencies

Background and project scope

Digital systems represent information with two stable values, 0 and 1. Logic gates combine these values according to Boolean algebra. Learners often meet the same rule in several forms: as a symbol in a circuit, an expression such as $Y = A \cdot B$, and rows in a truth table. The project is designed to connect those forms through direct interaction instead of asking a learner to infer the relationship from static notes alone.

The intended audience is first-year engineering students, teachers demonstrating introductory digital electronics, and hobbyists beginning circuit design. The scope is deliberately foundational: the app models standard combinational gates, adders, a multiplexer, an active-high SR latch, and a rising-edge D flip-flop. It is an educational simulation, not a hardware design or timing-analysis environment.

Learning interaction

1 SELECT	2 INPUT	3 OBSERVE	4 EXPLAIN
Choose a gate or circuit.	Toggle labeled 0/1 switches.	Follow the SVG signal and LEDs.	Read expression and truth row.

PROJECT FINDINGS

Combinational logic

Combinational components produce outputs from their current input values. In this simulator, every supported input combination is enumerated and paired with the output calculated by the corresponding Boolean function.

Finding 1 | Eight gates form the logic toolkit

GATE	BOOLEAN RULE	OUTPUT IN PLAIN LANGUAGE
AND	$A \cdot B$	1 only for 11
OR	$A + B$	1 for 01, 10, or 11
NOT	A'	Inverts A
NAND	$(A \cdot B)'$	0 only for 11
NOR	$(A + B)'$	1 only for 00
XOR	$A \oplus B$	1 when A and B differ
XNOR	$(A \oplus B)'$	1 when A and B match
Buffer	A	Copies A

For two-input gates, the table covers 00, 01, 10, and 11. NOT and Buffer each cover 0 and 1. As an input changes, JavaScript recomputes the result, colors active wires, updates the LED, and highlights the matching row. Gate symbols are drawn with inline SVG, so no image library is needed.

Finding 2 | Three combinational circuits connect gates to tasks

CIRCUIT	INPUTS	RULE	PURPOSE
Half adder	A, B	$Sum = A \oplus B$ $Carry = A \cdot B$	Adds two one-bit values.
Full adder	A, B, Cin	$Sum = A \oplus B \oplus Cin$ $Cout = AB + Cin(A \oplus B)$	Adds an incoming carry; 8 rows.
2-to-1 mux	A, B, Select	Select 0 → A Select 1 → B	Routes one selected data input.

Why the paired views matter

The simulator’ s main instructional value is the immediate cross-check. A learner can predict a gate output from its expression, toggle the inputs, then compare that prediction with both the LED and the highlighted truth-table row. CSV export makes the generated table reusable in worksheets or lab notes. The app also keeps input names and numeric values visible, so the signal colors are supplementary rather than the only way to read state.

Example: full-adder carry

When A=1, B=0, and Cin=1, Sum=0 and Cout=1. The result follows from $1 \oplus 0 \oplus 1 = 0$ and $(A \cdot B) + Cin(A \oplus B) = 0 + 1 \cdot 1 = 1$.

PROJECT FINDINGS / NEXT STEPS

State, implementation, and use

Finding 3 | Sequential circuits introduce memory

Unlike combinational logic, sequential circuits depend on stored state as well as current inputs. The application makes that distinction explicit in its characteristic tables by showing the prior state Q(t) and resulting state Q(t+1).

MODEL	INPUTS	CHARACTERISTIC	INTERACTIVE BEHAVIOR
SR latch	S, R	00: hold; 01: reset; 10: set; 11: invalid	The invalid row is marked X and the interface warns when both inputs are 1.
D flip-flop	D, Clock	On a rising edge, Q(t+1)=D; otherwise Q holds.	A manual Clock pulse demonstrates edge-triggered capture.

Implementation and user experience

The project has three core files. index.html defines the page and accessible controls; style.css supplies the responsive layouts, themes, and signal states; script.js contains the gate and circuit data, state transitions, truth-table generation, CSV construction, learning content, and quiz flow. The interface groups component selection, simulation, and reference information in adjacent panels, then stacks those panels on smaller screens.

The Learn tab explains the core vocabulary with short examples. The ten-question quiz gives one question at a time, provides answer feedback, and reports a final score. Light/dark mode, visible keyboard focus, labeled switches, status feedback, and mobile breakpoints support use in classroom demonstrations and individual practice.

Recommendations

NEXT STEP	OWNER / TIMING	SUCCESS MEASURE
Add browser-level regression checks	Project maintainer · next update	Run automated cases for gate vectors, state transitions, and CSV exports in a browser.
Introduce custom circuit building	Future feature · after core UX	Let learners connect gates; verify each constructed circuit against its generated table.
Add timing diagrams	Future feature · sequential module	Plot clock and state transitions so latch and flip-flop timing is visible over time.

Appendix | Verification and access

Source repository: github.com/Harshitsingh666/digital-electronics-simulator. The project can be opened directly from index.html or served with python -m http.server 8000, then visited at http://localhost:8000. It requires no build step or installed libraries.

Source-level verification covered all input rows for the eight gates, the half adder, full adder, and multiplexer; SR set, reset, hold, and invalid behavior; D flip-flop rising-edge capture; JavaScript

syntax; and CSV headers and row counts for all 13 selectable components. A live visual browser inspection was not part of that verification.